Summary of the Final Project: Automatic Dubbing System

(This project was conducted under the supervision of Dr. Noushin Riahi.)

The project initially commenced with the goal of creating an automatic dubbing system for English movies and series into Farsi, specifically targeting the ninth episode of the 'Friends' series. However, recognizing the challenges inherent in dubbing and considering the nuances of language, the project pivoted its focus towards educational courses in the field of computer science. This strategic shift was motivated by the potential impact on Iranian students who may lack proficiency in English, providing them access to educational content such as free data science courses available on platforms like YouTube.

Using Python and artificial intelligence (gender detection), the project successfully developed a program capable of receiving Farsi subtitle files and generating corresponding dubbed audio.

Conversion of Farsi Text to Speech:

One major challenge was implementing a program that could read Farsi sentences aloud. Given the lack of AI resources for the Farsi language, the implementation began with creating a Finglish (Farsi written in the Latin alphabet) dictionary at the guidance of the supervisor. While there are occasional issues with accurate pronunciation, efforts were made to optimize the results.

Workflow:

1. Input Processing:
   Extracting pure text from Farsi subtitle files was achieved through software. The Finglish dictionary, stored in a CSV file, was created word by word as the text was read.

2. Time Management:
   Time information from the original file was separated for adjusting audio speed, adding silences, and arranging sentences.

3. Dubbing Process:
   The subtitle file without diacritical marks and the Finglish dictionary were used to feed individual words into the Google Text-to-Speech (gtts) module, generating separate audio files based on gender detection from the time intervals.

4. Voice Modulation and Adjustment:
   Gender detection was employed on the original audio file using your provided timestamp information. Challenges included precise timing, but the current project version achieves an acceptable level of accuracy. Finally, speed adjustments and silences were applied to synchronize and create a cohesive dubbed audio file.

5. Conclusion and Challenges:
   The project, though providing a functional result, acknowledges several challenges and limitations. Notably, the use of subtitles for dubbing lacks the authenticity achieved by professional dubbers who simulate facial expressions and emotions. Additionally, the project recognizes the limitations of AI-generated speech in replicating human nuances, especially in high-speed scenarios.

6. Future Uses:
   Despite the mentioned limitations, the project envisions potential uses, such as automated dubbing for educational videos in the field of computer science. Students less proficient in English can benefit from dubbed content, providing an alternative to popular online courses available on platforms like YouTube, Coursera, and Udemy.

In conclusion, while the project addresses certain dubbing challenges, it emphasizes the irreplaceable role of human dubbing artists in capturing the true

essence of a performance. The system's future applications lie in scenarios where automated dubbing from subtitles suffices, particularly in educational contexts.

Functionality of Implemented Python Functions

Initially, we take a general look at the sequence of function operations:

```
import delete_panc
import save_dic
import from_file_to_dic
import add_delay
import extract_time
import speed_edjust_m
import gender_rec

#delete all the punction marks
delete_panc.delete()

# get the finglish qu and make a csv
save_dic.save()

#get the gender
gender_rec()

#open the csv and no_punct get every line in one audio
from_file_to_dic.dic_maker()

#extract time from sub-text file
extract_time.ex()

#for every audio get the right time using subtract_time func to edjust the speed
speed_edjust_m.speed()


#for every audio get the delay duration calling sub_time and getduration func and
finally concat them all
add_delay.make_audio()
```

1- Initially, we import the modules written in separate files into the main file

“delete_panc”

```
def delete():
 punctuations = '''!()-[]{}:"\،<>./؟@#$%^&*_~,و'''



# To take input from the user
# my_str = input("Enter a string: ")

# remove punctuation from the string
 no_punct = ""
#with open('sub-p-test.txt', "r" , encoding='utf-8') as f:
 f= open("sub-test.txt", "r" , encoding="utf-8")
 data=f.read()
 for char in data:
   #print(char)
   if char not in punctuations:
       no_punct = no_punct + char

# display the unpunctuated string
 with open('sub-no_punct.txt', 'w' , encoding="utf-8") as f1:
    f1.write(no_punct)
```

This module contains only one function. Initially, we define the punctuation marks we intend to remove as a string. Then, we define an empty string to place the text without punctuation marks and open the subtitle file. Using a loop for each character in the subtitle file, we check whether the character is an unwanted punctuation mark. If it is valid, it is added to the string we defined earlier. Finally, a new file named 'Subtitle Without Punctuation' is created, and the final string is placed in it.

Output:

sub-no_punct.txt

```
اه بر دیگه

هی عید شکر گزاری شمام هستا

می‌دنیبه جای فتیال نگاه کردن
می‌تنید کمك کنید

می‌دنیم

هی ریچل هدفت اینه که
مارشملها ر دایره دایره درست کنی

نهمانت هدفت دایره ای چیدنشنه

من میخام این کار بکنم

هر سال

می‌دنیبرای الین بار میخام
بشینم  سعی کنم یکی از این چیزها ببینم

نیمه دم

کی میخاد یه کم تپ بازی کنیم
```

“save_dic”

```
import io

def save():
 i=0
 thisdict = {}
 with open('sub-test.csv', encoding='utf-8') as f:
     for line in f:
            #break
            word=line.split(",")
            word[1]= str(word[1])
            word[1]=word[1].replace('\n','')
            thisdict[word[0]]=word[1]
 print(thisdict)
 words = []
 with open('sub-no_punct.txt', encoding='utf-8') as f:
     for line in f:
        for word in line.split():
            if word in thisdict.keys():
               continue
            else:
               finglish = input("enter the finglish equivelant for"+word +":")
               thisdict[word] = finglish

 with io.open('sub-test.csv','+a',encoding='utf-8-sig') as f:
    for k in thisdict:
      s = u''.join(k)
      s = u',.join([s,thisdict[k]]) + u'\n'
      f.write(s)
```

This module contains a single function that is tasked with creating a dictionary. This dictionary is designed to store the equivalent Finglish of each Persian word, where the key is the Persian word, and the value is its Finglish equivalent. Initially, the CSV dictionary file is opened, and if it doesn't exist (first run), it is created. Each entry in this file is then added to the dictionary.

Next, the subtitle file created in the previous step is opened, and each word is read. If these words are among the dictionary keys, the program continues without further action. However, if the equivalent of the word does not exist, it requests the equivalent from the user. In the initial version, this process was

automated, but after evaluation and comparison, it was determined that this automated process has a high error rate, and the generated words lack sufficient clarity. Therefore, this method was replaced. Finally, after completing the dictionary, all words are stored in a CSV file so that in each run, the dictionary becomes more comprehensive.

Output:

sub-test.csv

```
کاری,kari
که,ke
تسط,tavasote
به,be
انها,anha
داده,dade
شده,shode
را,ra
دنبال,donbal
میکنند,mikonand
اما,ama
چه,che
اتفاقی,etefaghi
می,mi
افتد,oftad
اگر,agar
بتاند,betavanand
یاد,yad
```

“gender_rec”

In this section of the project, a similar approach has been utilized from a GitHub repository: https://github.com/x4nth055/gender-recognition-by-voice

1- Initially, we need a dataset to train the network. In this project, the Mozilla Common Voice Dataset has been used.

2- Then, we install the required libraries.

3- We write a series of initial functions for storing and loading the data and labels.

4- It is intended to use a model with 5 hidden layers. Although it may not be optimal, it will work well for our goal with a very good accuracy percentage.

5- Next, we proceed to train the model. At this stage, the expectation is to find optimal weights.

6- Finally, we test the trained model."

```
                                    filename gender
66933  data/cv-valid-train/sample-171098.npy   male
66934  data/cv-other-train/sample-022864.npy   male
66935  data/cv-valid-train/sample-080933.npy   male
66936  data/cv-other-train/sample-012026.npy   male
66937  data/cv-other-train/sample-013841.npy   male
```

“from_file_to_dic”

```
import csv
import pandas as pd
import codecs
import persian
from gtts import gTTS
import os
import pyttsx3

def dic_maker():
 genderdic={}
 thisdict={}
 mytext=""
 z=1
# if word is not in dictioanry flag dose not change
 flag = 0
 with open('sub-test.csv', encoding='utf-8') as f:
     for line in f:
            #break
            word=line.split(",")
            word[1]= str(word[1])
            word[1]=word[1].replace('\n','')
            thisdict[word[0]]=word[1]

#delete("sub-no_punct.txt")
 file = open('gender.txt', 'r')
 #q count the line in genderdic
 q=0
 for line in file:
            genderdic[q]=line
            q= q+1
 with open('sub-no_punct.txt', encoding='utf-8') as f:
     for line in f:
        if line != '\n':
         for word in line.split():
            for k in thisdict:
              if word == k:
               #print(persian.convert_en_characters(word))
                mytext = mytext + thisdict[k] +"    "
         if mytext!="":
           print(genderdic)
           if genderdic[z-1] == "female\n":
            print("here")
            audio = gTTS(text=mytext, lang="en", slow=False)
            audio.save("f"+str(z)+".mp3")
            print("line:",z)
```

```
 else:
             engine = pyttsx3.init()
# We can use file extension as mp3 and wav, both will work
             engine.save_to_file(mytext, "f"+str(z)+".mp3")
# Wait until above command is not finished.
             engine.runAndWait()
         z=z+1
         mytext=""
         if z == 10:
             break
         continue
        for word in line.split():
            for k in thisdict:
              if word == k:
               flag = 1
               #print(persian.convert_en_characters(word))
               mytext = mytext + thisdict[k] +"    "
```

This module consists of a single function. In this function, we first open the Persian-to-Finglish dictionary file created in previous runs and load it into a dictionary. Then, we open the gender file created by the previous module to be able to generate Persian voices with the correct gender. After that, we open the subtitle file without punctuation marks and create a string for each line. Each string enters the voice generation stage based on gender. gTTS is used for female voices, and pytts is used for male voices. Each file is saved with its line number for future use in subsequent stages. Additionally, if a word in the Persian subtitle file is not in the dictionary, the user is prompted to enter its equivalent.

According to the implementer, the only weakness of the entire product is hidden in this function. If the Persian voice intonation is correct and the enunciated words are clear, the rest of the program and functions work correctly, and the film dubbing project progresses reasonably well (except for the discussed issues regarding the nature of dubbing work).

“extract_time”

```
import re
def ex():
# open the text file and read the data
 file = open("sub-with-time.txt",'r',encoding="utf-8")

 text = file.read()
# match a regex pattern for formatted dates
 matches = re.findall(r'(\d+:\d+:\d+)',text)

 file = open("only-time.txt" , 'w')
 file.write(" ")
 file.write(str(matches))
```

This module is very simple and consists of only one function, which is responsible for extracting timestamps from the original subtitle file. By extracting timestamps, two very important operations will be performed in the future. First, the interval of each dialogue is determined, and the speed of delivering each sentence can be calculated. Second, the silence time between dialogues is found. This function opens the subtitle file, then saves all strings that match the defined time pattern one by one in a file for later use by subsequent functions.

“speed_edjusment_m()”

For each audio file generated in the previous stages, the function speed_ed() is called by this module.

“speed_edjusment()”

```
import math
def convert_to_wav(i):
 import wave
 import sys
 from pydub import AudioSegment
 sound = AudioSegment.from_file(i)
##sound = AudioSegment.from_mp3(sys.argv[1])
 sound.export(i, format="wav")

#print(sys.argv[1])
def speed_ed(i,q):
 import soundfile as sf
 import pyrubberband as pyrb
 from pydub import AudioSegment
 from subtract_time import sub
 from get_duration import duration
 realtime=0
 cr_time=0
 speed=0
 convert_to_wav(i)
 realtime= int(duration(i))
 cr_time=str(sub(q))
 speed=(realtime/int(cr_time[6]))
 y, sr = sf.read(i)
 # Play back at extra low speed
 # Play back at 1.5X speed
 y_stretch = pyrb.time_stretch(y, sr, speed)
# Play back two 1.5x tones
 y_shift = pyrb.pitch_shift(y, sr, speed)
 sf.write(i, y_stretch, sr, format='wav')

 sound = AudioSegment.from_wav(i)
 sound.export(i, format="mp3")
```

This module contains two functions: one for adjusting the correct speed and the other for converting files from the default mp3 format to the wav format. This conversion is necessary because the methods used in the speed adjustment section only accept input in wav format.

In the speed adjustment section, two important variables are defined: one for the speed of the raw file, obtained using the get_duration() function, and the other for the desired speed, calculated by reading the time values and subtracting them using the subtract_time() function. Then, these two values are divided, and the speed of the file is adjusted using the stretch method and saved.

"Get_duration()"

```
import mutagen
from mutagen.wave import WAVE

# function to convert the information into
# some readable format
def audio_duration(length):
    hours = length // 3600 # calculate in hours
    length %= 3600
    mins = length // 60 # calculate in minutes
    length %= 60
    seconds = length # calculate in seconds

    return hours, mins, seconds # returns the duration

# Create a WAVE object
# Specify the directory address of your wavpack file
# "alarm.wav" is the name of the audiofile
def duration(i):
 audio = WAVE(i)
# contains all the metadata about the wavpack file
 audio_info = audio.info
 length = int(audio_info.length)
 #hours, mins, seconds = audio_duration(length)
 return length
```

For optimal code usage, this module is written with two functions. One of the functions takes an audio file as input, loads it, and passes the loaded file's duration as a raw parameter (in seconds) to the second function. The second function, by dividing the time into seconds, minutes, and hours,

returns the result to the first function. Then, the first function returns the result as an integer.

“subtract_time”

This module has a function responsible for time subtraction. To find the time of each dialogue, it is necessary to read the times from the file, and subtract them from the next entry. For example, the first dialogue is stated between times x and y, so y-x gives us the duration of the spoken sentence. In this function, we have an input that specifies for which dialogue we want to execute this function. Then, it reads the start time of that dialogue from the time file and considers it as the first time object. Next, it reads the next time, which is naturally the end time of the dialogue, and considers it as the second time object. At this point, both of our times are in string format since they are extracted from a text file. Therefore, it is necessary to convert them into a time format to subtract them accurately. This is done up to the .time() method, and then the times are subtracted from each other.

```
def sub(i):
 from datetime import datetime , date
 file = open("only-time.txt",'r')
 time_list = []

 lines = file.read().split(',')

 punctuations = ''''']!()-[]{}"\،<>./؟@#$%^&*_~,و'''

 no_punct = []
 nchar=''
 for char in lines:

   nchar=''
   #print(char)
   for z in char:
    if z not in punctuations:
        nchar=  nchar + z
   no_punct.append(nchar)

 #print(no_punct)
```

```
datetime_str1 = no_punct[i]
 datetime_str2= no_punct[i+1]
 #print(datetime_str1)
 #print(datetime_str2)
 #datetime_str1 = '00:00:04'
 #datetime_str2 = '00:00:05'
 if i == 0:
  datetime_object1 = datetime.strptime(datetime_str1,'%H:%M:%S').time()
 else:
  datetime_object1 = datetime.strptime(datetime_str1,' %H:%M:%S').time()

 datetime_object2 = datetime.strptime(datetime_str2,' %H:%M:%S').time()

#print(datetime_object1)  # printed in default format

 duration = datetime.combine(date.min , datetime_object2) - datetime.combine(date.min
, datetime_object1)
 #print(duration)
 return duration
```

“add_delay”

```
from pydub import AudioSegment
from pydub.playback import play
from os import path
from subtract_time import sub
from speed_edjust import speed_ed
from speed_edjust import convert_to_wav
import math
#for i in no_punct:
    #print(i)
def make_audio():
 z=1
 q=0
 final_song = AudioSegment.silent(duration=0)
 audio_out_file = "ex-edited.mp3"
 list=["f1.mp3","f2.mp3","f3.mp3","f4.mp3","f5.mp3","f6.mp3","f7.mp3","f8.mp3","f
9.mp3"]
```

```
 for i in list:
  audio_in_file = i
# create 1 sec of silence audio segment
  duration =str(sub(z))
 #print(duration[6])
  one_sec_segment =
AudioSegment.silent(duration=math.ceil(int(duration[6])*1000)-1000)  #duration in
milliseconds
  z=z+2

#read wav file to an audio segment
  song = AudioSegment.from_mp3(audio_in_file)

#Add above two audio segments
  final_song =  final_song + song + one_sec_segment
 #print(int(duration[6]))
 #play(final_song)
#Either save modified audi
 final_song.export(audio_out_file, format="mp3")
```

This module performs the final stages of the process within a single function. Firstly, it takes a list of audio files for which we adjusted the speed and gender in previous stages. Then, it defines an empty audio file, aiming to concatenate all the audio files along with silence between them into this single audio file. Initially, for each of the audio files, we determine the duration of silence after them using the subtract_time() function. Then, we append the original file along with the silence after it to the final audio file, and this process continues until the end. Finally, the audio file is saved.

This concludes the detailed description of the project stages and modules. I appreciate your attention and thank anyone who has taken the time to read this report.